

→ Stack (LIFO)

- A stack is a linear data structure operating on last in first out principle. Elements are added and removed from same end called as top / sp.
- It can be implemented using arrays and linked lists
- When implemented using array, it offers fixed size, contiguous memory and static allocation

→ Key applications of stack

1. Function call management
2. Expression parsing & conversion (infix to postfix / prefix evaluation)
3. Undo / Redo operations in text editors
4. Parenthesis / syntax matching in compilers.

→ Stack implementation

```
#include <stdio.h>
```

```
#define N 5
```

```
int stack[N];
```

```
int sp = -1;
```

```
void push (int i) {
```

```
if (sp == N-1) {
```

```
printf ("stack is full, can't enter");
```

```
}
```

```
else {
```

```
sp++;
```

```
stack[sp] = i;
```

```
printf ("you entered %d into stack", stack[sp]);
```

```
}
```

```

void pop () {
    if (sp == -1) {
        printf ("stack is empty, can't pop");
    }
    else {
        printf ("the removed element is %d\n", stack[sp]);
        sp--;
    }
}

```

```

int isEmpty () {
    if (sp == -1) {
        printf ("stack is empty");
    }
    else {
        printf ("stack is not empty");
    }
}

```

```

int peek () {
    if (sp == -1) {
        printf ("empty stack");
    }
    else {
        printf ("%d", stack[sp]);
    }
}

```

```

void display () {
    if (sp == -1) {
        printf ("empty stack");
    }
    else {
        for (i = 0; i <= sp; i++) {
            printf ("%d ", stack[i]);
        }
        printf ("\n");
    }
}

```

else {

printf("Stack elements (top to bottom) :-);

for (int i = sp; i >= 0; i--) {

printf("%d\n", stack[i]);

}

int main () {

push(5);

push(10);

push(11);

push(12);

push(90);

pop();

pop();

peek();

display();

}

main

main

main

main

main

main

main

main

main

main

main

main

main

main

main

main

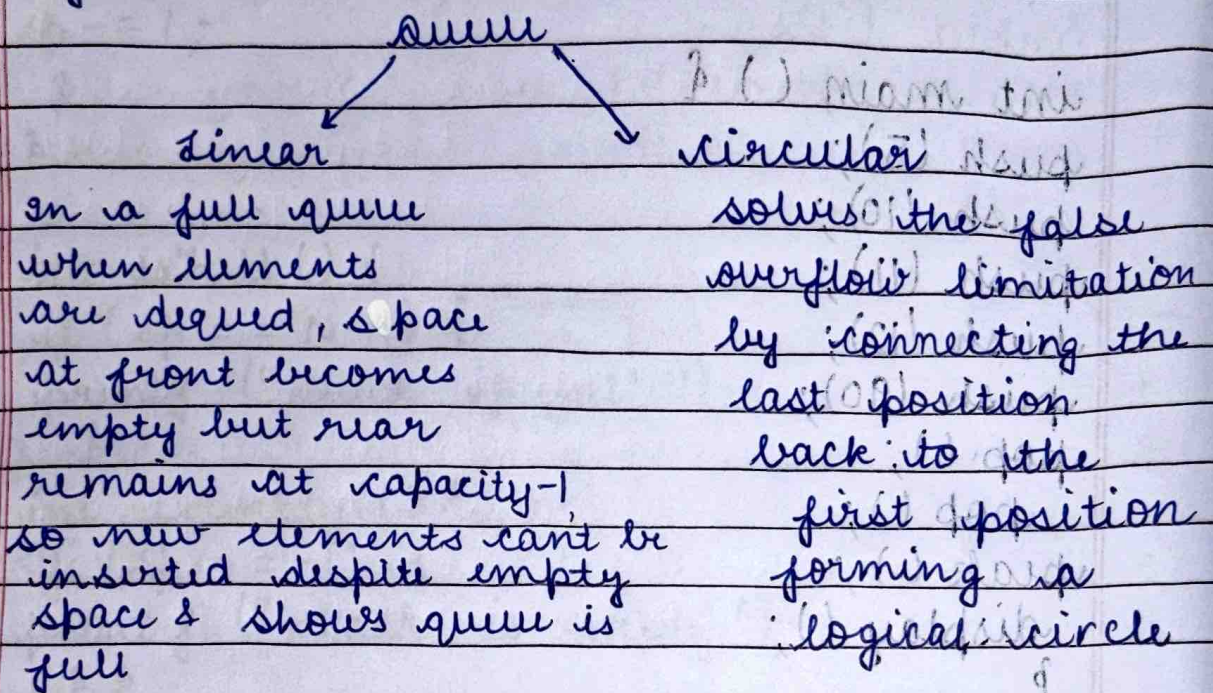
main

main

main

→ Queue (FIFO)

- A queue is a linear data structure that follows first in first out principle.
- Insertion happens at one end (rear) and deletion happens at opposite end (front)



→ Linear queues

```
# include <stdio.h>
```

```
# define N 5
```

```
int queue [N];
```

```
int front = -1;
```

```
int rear = -1;
```

```
int isFull() {
```

```
return rear == N-1;
```

```
} // End of isFull function
```

```
int isEmpty() {
    return front == -1 || front > rear;
}
```

as elements are dequeued
front $\uparrow$, if front > rear $\Rightarrow$ all
elements are dequeued

```
void enqueue(int value) {
    if (isFull()) {
        printf("Queue is full, can't enqueue");
        return;
    }
    if (front == -1) {
        front = 0;
    }
    rear++;
    queue[rear] = value;
    printf("enqueued: %d\n", value);
}
```

```
int dequeue() {
    if (isEmpty()) {
        printf("Queue is empty, can't dequeue");
        return -1;
    }
    int item = queue[front];
    front++;
    if (front > rear) {
        front = -1;
        rear = -1;
    }
    return item;
    printf("dequeued: %d", item);
}
```

```

int peek () {
    if (isEmpty()) {
        printf ("Queue is empty");
        return -1;
    }
    return queue [front];
}

```

```

void display () {
    if (isEmpty()) {
        printf ("empty!!");
        return;
    }
    printf ("Queue contents");
    for (int i = front; i <= rear; i++) {
        printf ("%d", queue [i]);
    }
    printf ("\n");
}

```

```

int main () {
    enqueue (10);
    enqueue (20);
    enqueue (30);
    display ();
    dequeue ();
    peek ();
    return 0;
}

```

• (Note: "b.x : kuyup") f... ; note: number

→ Circular Queue

```
# include <stdio.h>
```

```
# define N 5; ("array to store")
```

```
int cir-queue [N];
```

```
int front = -1;
```

```
int rear = -1; ("array")
```

```
int isFull () { (rear + 1 == front) }
```

```
return (rear + 1) % N == front;
```

```
}
```

```
int isEmpty () {
```

```
return front == -1;
```

```
}
```

```
void enqueue (int value) {
```

```
if (isFull()) { ("b.x : kumar") }
```

```
printf ("Queue is full");
```

```
return;
```

```
}
```

```
if (isEmpty()) {
```

```
front = 0; ("array")
```

```
rear = 0;
```

```
}
```

```
else {
```

```
rear = (rear + 1) % N;
```

```
}
```

```
cir-queue [rear] = value; ("b.x")
```

```
printf ("Enqueued : %d", value);
```

```
}
```

```
printf ("r/n") }
```

```

int dequeue () {
    if (isEmpty ()) {
        printf ("Queue is empty");
        return -1;
    }
    int data = queue [front];

```

```

    if (front == rear) {
        front = -1;
        rear = -1;
    }

```

```

    else {
        front = (front + 1) % N;
        printf ("dequeued : %d", data);
    }

```

```

int peek () {
    if (isEmpty ()) {
        printf ("empty queue");
        return -1;
    }
    printf ("elements :");
    int i = front;
    while (1) {
        printf ("%d", queue [i]);
        if (i == rear) break;
        i = (i + 1) % N;
    }
    printf ("\n");
}

```

```

int main () {
    enqueue (10);
    enqueue (20);
    enqueue (30);
    enqueue (40);
    enqueue (50);
    display ();
    dequeue ();
    dequeue ();
    display ();
    peek ();
    return 0;
}

```

→ Applications of queue

1. CPU scheduling
 - First come, first served & round robin algorithms
2. Print Spooling
 - Prints multiple documents according to order of commands given
3. Network Buffering
 - To store incoming data packets in routers & switches
4. Disk Input/output Handling
 - Implement read & write in operating systems
5. Graph Algorithms
 - Breadth first search graph traversal to do line-wise traversal of graphs

→ expression notations

1. Infix

- operator lies between the operands
- eg $\rightarrow A + B * C$

2. Postfix (Reverse Polish notation)

- operator used after operand
- eg $\rightarrow ABC * +$

3. Prefix (Polish notation)

- operator used before operand
- eg $\rightarrow +A * BC$

→ Conversion of infix to postfix using stack

Operator

Associativity

()	Linear	Left to right
-----	--------	---------------

^

Right to left

* /

Left to right

+ -

Left to right

• Rules

- If the incoming operator is having higher priority than top of the stack operator, so incoming operator resides over already existing operator in stack
- If the incoming operator is having lower priority than top of the stack operator so higher priority is popped out and lower priority pushed in
- If the incoming operator is having equal priority to the top of the stack, existing

Q $A + B * (C - D) \wedge E \wedge F / (G + H * I)$ convert into postfix notation

get the expression and convert it into postfix notation. $i = 0$ two stacks are used. Postfix notation is used. A is the postfix notation. A is the postfix notation.

+

+

A

mi

*

AB

AB

(

+

AB

C

(-)

*

ABC

-

+

ABC

D

+

ABCD

$0 = 1$

)

+

ABCD -

E

+

ABCD -

+

F

+

ABCD - E

d

A

+

ABCD - E

*

F

+

ABCD - EF

)

/

+

ABCD - EF A *

(

+

ABCD - EF A *

G

+

ABCD - EF A * G

+

+

ABCD - EF A * G

H

+

ABCD - EF A * G H

*

+

ABCD - EF A * G H

I

+

ABCD - EF A * G H I

)

+

ABCD - EF A * G H I *

ABCD - EF A * G H I * +

Q $A + B * - C ^ (D / - E ^ F)$ it's postfix

initially $i=0$ stack postfix

push A, push B, push +, push -, push C, push ^, push (, push D, push /, push -, push E, push ^, push F, push)

pop F, pop ^, pop E, pop -, pop /, pop D, pop ^, pop C, pop -, pop B, pop +, pop A

AB

AB*

AB*+

AB*+C

AB*+C

AB*+C

AB*+C

AB*+CD

AB*+CD

AB*+CD/

AB*+CD/E

AB*+CD/E

AB*+CD/E

AB*+CD/E^

AB*+CD/E^A

AB*+CD/E^A-

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

Ans →

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^

AB*+CD/E^A-^